



Data Structure

(Course Code: 25B11CI318)

Threaded Binary Trees

Mr. Sandeep Kumar Patel

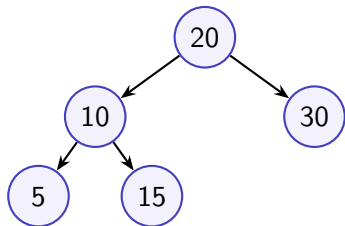
Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

Introduction

A linked binary tree stores two child-pointer fields in every node.



For a Non-Empty Tree

With N nodes:

$$\text{Pointer fields} = 2N$$

$$\text{Child links} = N - 1$$

$$\text{NULL fields} = N + 1$$

Example: $N = 5$ gives 10 pointer fields, 4 child links and 6 NULL fields.

Definition of Threaded Binary Tree

Definition

A **threaded binary tree** reuses selected missing-child pointer fields as **threads** that support a specified traversal order.

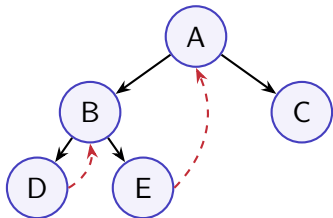
For **inorder threading**:

- A left thread points to the inorder predecessor.
- A right thread points to the inorder successor.
- Existing child links remain unchanged.

Convention Used in These Slides

Unless a header is explicitly shown, the first node's predecessor and the last node's successor are NULL.

Why Do We Need Threaded Binary Trees?



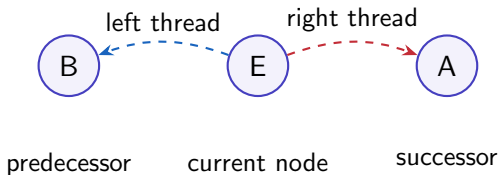
- After visiting *D*, inorder must return to *B*.
- After visiting *E*, it must continue at *A*.
- Threads store these return links directly.

D, B, E, A, C

Solid black: child link. Blue dashed: predecessor thread. Red dashed: successor thread.

What is a Thread?

A thread is a traversal link, not a parent–child edge.



Example

In the sequence D, B, E, A, C , node E has predecessor B and successor A .

These links occupy E 's pointer fields only because E has no actual children.

Inorder Predecessor and Successor

Consider the inorder sequence

10, 20, 30, 40, 50.

Predecessor of a Node

If a left child exists, take the rightmost node in that left subtree.

Otherwise, follow the left thread.

Successor of a Node

If a right child exists, take the leftmost node in that right subtree.

Otherwise, follow the right thread.

$\text{Pred}(30) = 20,$ $\text{Succ}(30) = 40.$

Inorder neighbors are sorted neighbors only when the underlying tree satisfies the BST ordering property.

Types of Threaded Binary Trees

① Single-threaded

- Left-threaded: missing left links store predecessors.
- Right-threaded: missing right links store successors.

② Double-threaded

- Both missing-left and missing-right fields are threaded.

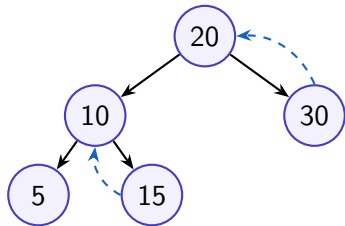
Traversal Direction

Right threading supports forward inorder traversal.

Left threading supports reverse inorder traversal.

Double threading supports both.

Left Threaded Binary Tree



Left Threads

$15.\text{left} = 10,$ $30.\text{left} = 20$

The first inorder node has:

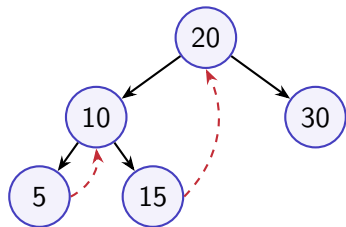
$5.\text{left} = \text{NULL}.$

Only missing left-child fields store predecessor threads. Right fields retain ordinary child/NULL meanings.

Reverse inorder: 30, 20, 15, 10, 5

Solid black: child link. Blue dashed: predecessor thread. Red dashed: successor thread.

Right Threaded Binary Tree



Right Threads

$5.right = 10,$ $15.right = 20$

The last inorder node has:

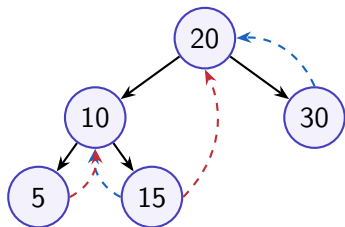
$30.right = \text{NULL}.$

Only missing right-child fields store successor threads. Left fields retain ordinary child/NULL meanings.

Forward inorder: 5, 10, 15, 20, 30

Solid black: child link. Blue dashed: predecessor thread. Red dashed: successor thread.

Double Threaded Binary Tree



- Missing left child: predecessor thread.
- Missing right child: successor thread.
- Supports traversal in either inorder direction.

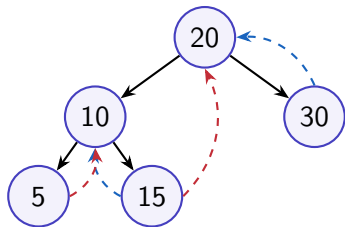
Boundary Links

Here, 5.*left* = NULL and 30.*right* = NULL.

Their flags still indicate **thread fields**.

Solid black: child link. Blue dashed: predecessor thread. Red dashed: successor thread.

Example of Double Threaded Binary Tree



Node	Left thread	Right thread
5	NULL	10
10	—	—
15	10	20
20	—	—
30	20	NULL

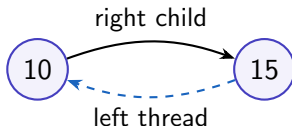
— means the field is an actual child link.

Inorder: 5, 10, 15, 20, 30

Solid black: child link. Blue dashed: predecessor thread. Red dashed: successor thread.

Child Link vs Thread

The pointer value alone does not reveal its meaning.



Flag Convention

0 = actual child,	1 = thread
-------------------	------------

For the links above:

$$10.rthread = 0, \quad 15.lthread = 1.$$

Never recursively follow a thread as if it were a child: threads can lead back to ancestors.

Node Structure

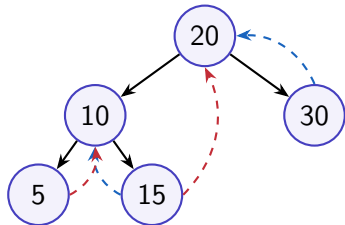
```
struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
    int lthread; /* 0: child, 1: predecessor */  
    int rthread; /* 0: child, 1: successor */  
};
```

Interpretation

A flag value of 1 means there is no actual child on that side. The pointer stores an inorder neighbor, or NULL at a boundary.

The following code assumes a double-threaded tree using this convention and no header node.

Example of Thread Representation



Node 15

$left = 10, \quad lthread = 1,$
 $right = 20, \quad rthread = 1.$

Both links are threads.

Node 10

$left = 5, \quad lthread = 0,$
 $right = 15, \quad rthread = 0.$

Both links are child links.

Normal Inorder Traversal

Ordinary inorder traversal follows:

Left subtree $\rightarrow$ Root $\rightarrow$ Right subtree.

- 1 Traverse the left subtree.
- 2 Visit the current node.
- 3 Traverse the right subtree.

Returning to Ancestors

The usual recursive method uses the call stack. The usual iterative method uses an explicit stack: $O(h + 1)$ auxiliary space.

Other methods, such as Morris traversal, can use constant space by temporarily modifying links. Permanent threading stores traversal links as part of the representation.

Inorder Traversal in a Threaded Tree

- ① Start at the leftmost node.
- ② Visit the current node.
- ③ If its right field is a thread, follow that thread.
- ④ Otherwise, move to the leftmost node of its actual right subtree.
- ⑤ Stop when the current pointer becomes NULL.

Successor Rule

$$\text{Succ}(x) = \begin{cases} x.\text{right}, & x.\text{rthread} = 1, \\ \text{LeftMost}(x.\text{right}), & x.\text{rthread} = 0. \end{cases}$$

Finding the Leftmost Node

```
struct Node *leftMost(struct Node *node)
{
    if (node == NULL)
        return NULL;

    while (node->lthread == 0)
        node = node->left;

    return node;
}
```

Why Check the Flag?

A non-NULL left pointer may be a predecessor thread. Descend only through actual left-child links.

Inorder Traversal Algorithm

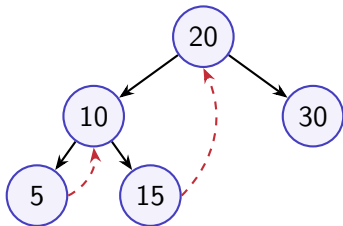
```
void inorder(struct Node *root)
{
    struct Node *current = leftMost(root);

    while (current != NULL) {
        printf("%d ", current->data);

        if (current->rthread == 1)
            current = current->right;
        else
            current = leftMost(current->right);
    }
}
```

Requires `stdio.h`, the node structure, and `leftMost`. An empty tree produces no output.

Traversal Example



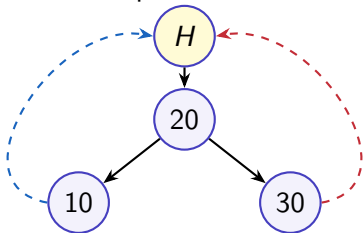
Movement	Reason
Start at 5	Leftmost node
5 → 10	Right thread
10 → 15	Right subtree's leftmost
15 → 20	Right thread
20 → 30	Right subtree's leftmost
30 → NULL	Boundary thread

5, 10, 15, 20, 30

Solid black: child link. Blue dashed: predecessor thread. Red dashed: successor thread.

Header Node

A header H can replace NULL boundary links.



- $H.left$ points to the root.
- First node's left thread points to H .
- Last node's right thread points to H .
- Traversal stops at H ; do not visit it.

One convention uses $H.left = H$ for an empty tree. Header fields and flags must follow the chosen implementation.

Advantages of Header Node

- A single sentinel represents both inorder boundaries.
- Traversal ends when the current pointer equals H .
- The root has a consistent access point through $H.left$.
- Empty-tree and root updates can use the same sentinel.
- Boundary cases need fewer separate NULL checks.

Consistency Matters

The previous C traversal uses NULL termination. With a header, change initialization, empty-tree handling, and termination consistently.

Insertion in a Threaded Binary Tree

Insert a new leaf X as the left child of P .

Precondition

$P.lthread = 1$: P has no actual left child. Its old left field stores its predecessor.

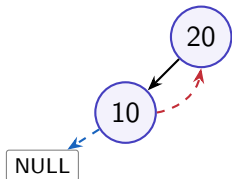
$$\begin{aligned} X.left &= P.left, & X.lthread &= 1, \\ X.right &= P, & X.rthread &= 1, \\ P.left &= X, & P.lthread &= 0. \end{aligned}$$

Scope

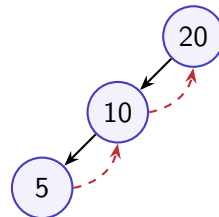
These updates fill an empty child position. They must not overwrite an existing subtree. For a threaded BST, first locate the correct position by key.

Example of Left Insertion

Before



After inserting 5



$5.left = \text{NULL}, \quad 5.right = 10, \quad 10.lthread = 0.$

Solid black: child link. Blue dashed: predecessor thread. Red dashed: successor thread.

Right Insertion

Precondition

$P.rthread = 1$: no actual right child.

$X.right = P.right,$

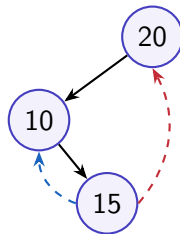
$X.left = P,$

$X.lthread = X.rthread = 1,$

$P.right = X,$

$P.rthread = 0.$

The new node inherits the old successor and uses P as its predecessor.



Insert 15 as the right child of 10.

Deletion in Threaded Binary Trees

Count **actual children using flags**, not non-NULL pointers.

- ① **No children:** both flags are 1.
- ② **One child:** exactly one flag is 0.
- ③ **Two children:** both flags are 0.

What Must Be Preserved?

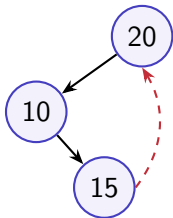
The parent/root link must remain valid, and no surviving child link or thread may point to the deleted node.

The following cases assume a double-threaded BST. Root deletion also requires updating the root pointer.

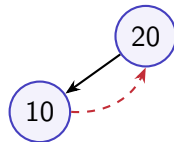
Deletion: Leaf Node

Delete node 15, the actual right child of 10.

Before



After



$$10.\text{right} = 15.\text{right} = 20, \quad 10.\text{rthread} = 1.$$

For a left leaf X of P , use $P.\text{left} = X.\text{left}$ and $P.\text{lthread} = 1$.

Deletion: One or Two Children

One Actual Child

Reconnect the parent directly to the child.

If X has only a left child, its subtree's rightmost node must thread to $\text{Succ}(X)$.

If X has only a right child, its subtree's leftmost node must thread to $\text{Pred}(X)$.

Two Actual Children

- 1 Find S , the leftmost node in the right subtree.
- 2 Copy S 's key and associated payload into X .
- 3 Delete S , which has no actual left child.
- 4 Apply the leaf or one-child repair.

Copy data, not structural pointers or thread flags.

Time and Space Complexity

For a complete inorder traversal of N nodes:

$$\text{Time} = \Theta(N), \quad \text{Auxiliary space} = O(1).$$

- Each node is visited once.
- Across the traversal, child descents and thread movements total $O(N)$.
- A single successor query may take $O(h + 1)$.
- Successive inorder advances cost $O(1)$ amortized.

Storage

Constant auxiliary traversal space does not mean constant tree storage: nodes and flags still occupy $O(N)$ space.

Normal vs Threaded Binary Tree

Feature	Ordinary Binary Tree	Inorder-Threaded Tree
Missing-child fields	Usually NULL	Neighbor threads or boundaries
Standard inorder	Recursion or explicit stack	Thread-guided iteration
Successor	Needs navigation/state	Direct if right field is a thread
Predecessor	Needs navigation/state	Direct if left field is a thread
Link tags	Usually unnecessary	Distinguish children from threads
Traversal space	$O(h + 1)$ for standard DFS	$O(1)$
Updates	Maintain child links	Maintain child links and threads

Comparison concerns conventional traversal implementations; constant-space alternatives also exist for ordinary trees.

Single vs Double Threaded Tree

Feature	Single-Threaded	Double-Threaded
Threaded fields	Only left or only right	Both missing-child sides
Left threading	Predecessor links only	Predecessor links included
Right threading	Successor links only	Successor links included
Natural iteration	One inorder direction	Both inorder directions
Direct neighbor	When corresponding thread exists	When corresponding thread exists
Maintenance	One type of thread	Both types of thread

Advantages of Threaded Binary Trees

- Reuses pointer fields that would otherwise be NULL.
- Supports inorder traversal without recursion.
- Avoids an explicit traversal stack.
- Provides direct neighbor access when a thread exists.
- Double threading supports reverse inorder traversal.
- Supports constant-space iteration across repeated scans.

Auxiliary traversal space = $O(1)$

Disadvantages

- Insertion and deletion must preserve thread invariants.
- Flags consume storage and must remain consistent.
- Ordinary recursive tree code cannot follow all pointers.
- Rotations and structural changes need thread-aware code.
- A single neighbor query is not always constant-time.
- Threading alone does not balance a BST.

Implementation Trade-Off

Frequent updates increase maintenance complexity. Threading does not automatically improve update-time bounds.

Applications

Threading is useful when inorder iteration is important:

- Ordered-set and dictionary iterators using threaded BSTs.
- Repeated forward or reverse scans.
- Tree processing with limited traversal-stack space.
- Environments where recursive traversal is undesirable.
- Inorder inspection of expression or syntax trees.

Choosing the Representation

Use threading when its iteration benefits justify the extra care required for structural updates.

Important Numerical Property

For a non-empty linked binary tree with N nodes:

$$\text{Child-pointer fields} = 2N$$

$$\text{Actual child links} = N - 1$$

$$\text{Missing-child fields} = N + 1$$

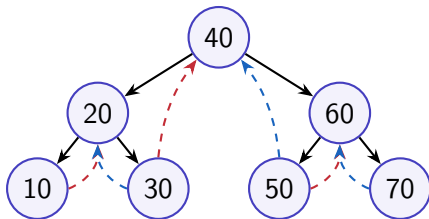
After Complete Double Threading

With NULL boundary convention:

$$N - 1 \text{ non-NULL threads} + 2 \text{ NULL boundaries}$$

With a header, those two boundaries point to the header. This count excludes the header's own fields.

Solved Example



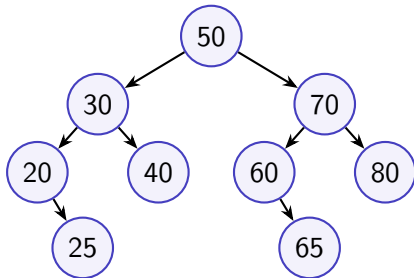
10, 20, 30, 40, 50, 60, 70

Node	Left thread	Right thread
10	NULL	20
30	20	40
50	40	60
70	60	NULL

Nodes 20, 40, 60 have actual children on both sides.

Solid black: child link. Blue dashed: predecessor thread. Red dashed: successor thread.

Practice Question

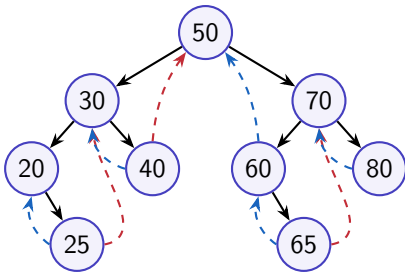


Questions

- 1 Write the inorder sequence.
- 2 Find predecessors and successors of 25, 40, 60, 65.
- 3 Add all inorder threads.
- 4 Distinguish threads from child links.

Practice Question: Answer

20, 25, 30, 40, 50, 60, 65, 70, 80



Node	Pred.	Succ.
25	20	30
40	30	50
60	50	65
65	60	70

20.*left* = NULL and 80.*right* = NULL.

Important: 60.*right* = 65 is a child link, not a thread.

Solid black: child link. Blue dashed: predecessor thread. Red dashed: successor thread.

Quick Revision

Threaded Binary Tree

A binary tree plus traversal links in missing-child fields, with tags that distinguish the two link types.

- Left thread: inorder predecessor.
- Right thread: inorder successor.
- A thread never replaces an actual child.
- Single threading supports one inorder direction.
- Double threading supports both directions.
- Check flags before descending through pointers.

Summary

Left thread = inorder predecessor link

Right thread = inorder successor link

Complete inorder traversal time = $\Theta(N)$

Auxiliary traversal space = $O(1)$

Key Idea

Threading stores traversal information in the tree itself. It removes the need for a traversal stack, while requiring careful maintenance during structural updates.

Thank You